# tierlm 0.5.0

A performance-first local LLM engine: every part of the model in the right memory (VRAM, RAM and soon SSD), so big models run fast and your RAM, not your GPU, sets the limit

| | |
|---|---|
| **Version** | 0.5.0 (October 29, 2026) |
| **What it is** | A from-scratch C++ inference engine with a web chat and live dashboard. Windows and Linux; Intel, AMD and NVIDIA GPUs. |
| **Built by** | Written by Claude (Anthropic's AI), with Chad Jameson (EHTi) directing the project, making the design calls, and testing it on real hardware. |
| **Licence** | MIT. Copyright (c) 2026 Chad Jameson (EHTi). |
| **Source** | github.com/EHTi-Chad/tierlm |

### Contents

1. What tierlm is · 2. What's new in 0.5.0 · 3. Supported models · 4. Getting started · 5. The settings · 6. Connect other apps · 7. How it works · 8. Test results · 9. Troubleshooting · 10. Building from source · 11. Known limits and what's next

## 1. What tierlm is

Large language models are usually run on servers with hundreds of gigabytes of graphics memory. At home, the graphics card (GPU) is the fast part but has the least memory: 8 GB on a typical laptop card, 32 GB on the biggest consumer card. System RAM is bigger but several times slower, and an SSD is bigger again and slower still.

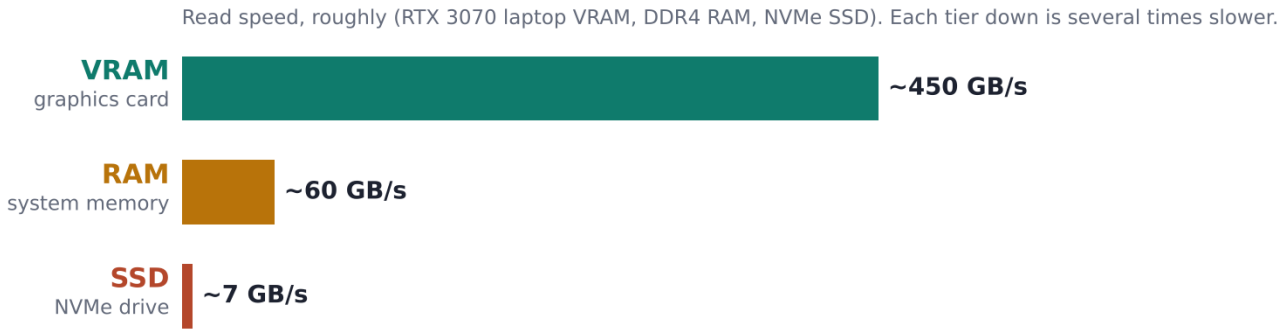



*Figure 1. The three memory tiers. tierlm puts the parts of a model that are used most in the fastest tier that has room.*

**tierlm** loads a model in the common GGUF format, works out where every piece of it should live, and runs it. If the model fits on the GPU, it runs there. If it doesn't, tierlm splits it: as much as fits goes into VRAM and the rest into RAM, where the CPU works on it. The idea comes from **Strata**, an open-source engine that runs a 125-billion-parameter model on a 12 GB GPU. tierlm is its own engine, built on the same principle:

- **Fast**: C++, with the ggml library's math kernels for CPU, Vulkan and CUDA.

- **Reads common models**: the Llama, Mistral, Qwen and OLMoE families, straight from the files people download from Hugging Face or already have in LM Studio.
- **Tiered memory**: VRAM first, then RAM, then SSD, decided automatically.
- **Nothing to install**: unzip, double-click, and pick a model in the browser. The only requirement is a normal graphics driver.

tierlm does **not** use llama.cpp. The only outside engine code is ggml, a small MIT-licensed library of fast matrix kernels and the quantized file formats; tierlm changes it in two small places (50 lines), all listed and explained in docs/GGML_CHANGES.md. The tokenizers, the model graphs, the memory planning, the tier loader, the server and the web app are all tierlm's own.

## 2. What's new in 0.5.0

0.5.0 makes tierlm about 50% faster on NVIDIA cards, can keep a model's busiest experts in VRAM, and documents every change it makes to ggml.

| Area | New in 0.5.0 |
|---|---|
| CUDA graphs | On NVIDIA cards, each word's GPU work is now recorded once and replayed with a single launch instead of thousands. On an 8 GB laptop RTX 3070, Qwen3.6-35B-A3B with experts in RAM went from 21.7 to 33.3 tokens/second, and the fastest setting from 31.6 to 35.3. The answers are exactly the same; only how the work is launched changed. |
| Hot experts | tierlm learns which experts each mixture-of-experts model uses as you chat, and can keep those individual experts in VRAM instead of whole expert layers. Whether that's faster depends on the PC (on the laptop the plain split still won), so it's off by default: Find the fastest settings measures both and turns it on where it wins. Model info shows how many experts are in VRAM and the share of the work they did. |
| Open about ggml | tierlm's two small changes to ggml (the CPU skips experts computed elsewhere; an optional timing switch) are described line by line in docs/GGML_CHANGES.md, with a patch file for other projects, and published as a fork. The page also explains what it took to make CUDA graphs work and a CUDA limit that crashes when a token lists the same expert twice. |
| Timing switch | With TIERLM_PROFILE=1, tierlm prints where each word's time goes: preparing it, and per device the copying and the work. That's how the slow GPU launches were found. |
| Fixes | A crash (illegal memory access) on NVIDIA cards with hot experts. Find the fastest settings now saves hot experts only when they measured fastest. |
| Testing | Every model type, split between one or two stand-in GPUs and RAM, by layers, by experts and with hot experts at many VRAM sizes, matches the reference (about 1e-6), including long prompts and speculative decoding. CUDA graphs and the hot-expert split were checked on the RTX 3070 laptop. |

## From 0.4.0

| Area | From 0.4.0 |
|---|---|
| Automatic split | Mixture-of-experts models that don't fit in VRAM now keep attention, the router and the context on the GPU, with only the experts in RAM; dense models still move whole layers. On an 8 GB laptop RTX 3070, Qwen3.6-35B-A3B went from 9.9 to 17 tokens/second from this alone (0.4.0). |
| Speculation defaults | Guesses per check defaults to 2 for MoE models (each guess pulls in its own experts from RAM) and 3 for dense ones. A 1-guess option. With both changes, Qwen3.6-35B-A3B on the laptop reached 22.3 tokens/second, 2.25 times where it started. |
| Find the fastest settings | One button on the setup screen loads the model and measures the settings that only change speed on this PC: the split mode, CPU threads, and speculative decoding with 1 to 4 guesses. Each result appears as it's measured; the fastest is kept and saved for the model. |
| Several GPUs | Two or more cards of the same kind (several NVIDIA cards with CUDA, say) become VRAM tiers: the model fills the first, then the next, then RAM. The setup screen shows a VRAM bar per card. |
| New chat layout | A side menu that folds away: Settings (sampling, speculative decoding on and off, a system prompt), API, a Console showing what the engine is doing, Model info, saved Chats and system-prompt presets, and a one-click Benchmark. A thinner live view on the right, and an Auto / Light / Dark theme switch. |
| Expert map, explained | "Which experts are working" has a ? that explains it in plain words, a Sorted view (each layer's busiest experts first), a strip showing which layers' experts are in VRAM or RAM, hover details, and a full-size view. It now also works with speculative decoding on. |
| Smaller things | A loading progress bar. A steadier speed chart with speculative decoding (tokens arrive in bursts). Requests that arrive just as a reply finishes wait a moment instead of failing. |
| Testing | Every model type split over two stand-in GPUs and RAM at many memory sizes matches the reference, including speculative decoding (140 checks). The expert map gets exactly the same picks with and without speculation. Auto-tune, benchmark and the new layout checked in a browser, in both themes and at phone width. |

# 3. Supported models

tierlm reads the architecture from the file. The setup screen marks every model as supported or says why not.

| Family | GGUF architecture | Examples | Status |
|---|---|---|---|
| OLMoE | olmoe | OLMoE-1B-7B-0924-Instruct | Tested on real hardware |
| Qwen3 | qwen3 | Qwen3-0.6B to Qwen3-32B | Tested: Qwen3-0.6B |
| Qwen3 MoE | qwen3moe | Qwen3-30B-A3B, Qwen3-235B-A22B | Tested: Qwen3-30B-A3B |
| Llama 3 / 3.1 / 3.2 / 3.3 | llama | Llama-3.2-1B, Llama-3.1-8B, Llama-3.3-70B | Tested: Llama-3.2-1B |
| Llama 2, Mistral 7B, TinyLlama | llama (SentencePiece) | Mistral-7B-Instruct, TinyLlama-1.1B-Chat | Tested: TinyLlama-1.1B |
| Mixtral | llama (with experts) | Mixtral-8x7B | Math verified; not yet run on real weights |
| Qwen2 / 2.5 | qwen2 | Qwen2.5-7B-Instruct, Qwen2.5-Coder | Math verified; not yet run on real weights |
| Qwen3.5 / 3.6 / 3.8 dense (hybrid) | qwen35 | Qwen3.6-27B, Qwen3.8-27B, Qwen3.5-9B | Tested: Qwen3.6-27B, with speculative decoding |
| Qwen3.5 / 3.6 MoE (hybrid) | qwen35moe | Qwen3.6-35B-A3B, Qwen3.5-122B-A10B | Tested: Qwen3.6-35B-A3B Q8_0 |
| Qwen3-Next (hybrid MoE) | qwen3next | Qwen3-Coder-Next, Qwen3-Next-80B-A3B | Math verified; not yet run on real weights |
| Gemma, Phi, DeepSeek, gpt-oss, ... | various | Gemma 4, gpt-oss-20b | Not yet |

Models are often quantized by different people (bartowski, Unsloth, the model makers themselves); all of them work as long as the family is supported, including Unsloth's mixed "dynamic" quants. Big models that come as several files (`-00001-of-00003.gguf` ...) load as one model: put all the parts in the same folder. A file called `mmproj-*.gguf` next to a model is its **vision add-on** (for reading images); tierlm lists it, but doesn't use images yet. A small file whose name starts with `mtp-` is a **draft add-on**: keep it in the same folder as its model and tierlm uses it for Speculative decoding (the setup screen labels it).

# 4. Getting started

This is everything a new user needs. No programming, no Python, no CUDA toolkit, no SDKs.

## What you need

| Item | Details |
|---|---|
| Windows 10 or 11, 64-bit | Any 64-bit Intel or AMD processor; tierlm uses the fastest instructions it has (AVX2, AVX-512...). Linux works too, built from source. |
| System RAM | At least 32 GB to run models bigger than your GPU (what tierlm is for); 64 GB or more recommended. The model must fit in VRAM and RAM together with about 8 GB to spare: 8 GB GPU + 32 GB RAM runs models up to about 30 GB, 8 + 64 GB about 60 GB, 32 + 64 GB about 85 GB. |
| A graphics driver | A current NVIDIA, AMD or Intel driver. Nothing else: NVIDIA's CUDA runtime is included. Without a GPU, tierlm runs on the CPU. |
| The tierlm zip | From the Releases page. **tierlm-0.5.0-windows.zip** (about 550 MB) has everything, including NVIDIA's CUDA engine: get it for an NVIDIA RTX card. **tierlm-0.5.0-windows-lite.zip** (about 20 MB) has only the Vulkan engine, for Intel Arc, AMD Radeon or no GPU (it also runs on NVIDIA, slower). Both pick the fastest engine they have by themselves. |
| A model file (.gguf) | From Hugging Face, or the ones you already have in LM Studio. |

## Step by step

1. **Download** `tierlm-0.5.0-windows.zip` (NVIDIA RTX) or `tierlm-0.5.0-windows-lite.zip` (everything else) from github.com/EHTi-Chad/tierlm/releases. Right-click it, choose **Extract All...**, and extract it somewhere simple, such as `C:\tierlm`.
2. **Get a model.** If it comes as several files (`-00001-of-00003.gguf` ...), download all of them. To see tierlm do what it's for, download a model bigger than your GPU, such as `Qwen3-30B-A3B-Q3_K_M.gguf` (14.7 GB; 27 tokens/s on an 8 GB laptop RTX 3070). Mixture-of-experts models (A3B, A10B...) stay fast with most of the model in RAM. For a quick first check, `Llama-3.2-1B-Instruct-Q4_K_M.gguf` (0.8 GB) works anywhere. Put the file in the `models` folder inside tierlm. If you already have models elsewhere (LM Studio's folder, for example), leave them there: step 4 points tierlm at them.
3. **Start it.** Double-click `start-tierlm.bat`. If Windows shows *Windows protected your PC*, click **More info**, then **Run anyway** (tierlm isn't signed with a paid certificate yet). A black window opens: leave it open while you use tierlm. Your browser opens `http://127.0.0.1:8040`.
4. **Pick a model** on the setup screen. **Detected hardware** shows your GPU and CPU and which one will be used. A **VRAM** bar shows how the GPU's memory will be used: the model, its context (KV cache), working memory, other programs, and what's free; a **System RAM** bar shows any part that goes in RAM. **GGUF models** lists the models in your folder; the **⋯** menu lets you **Choose models folder...** (a normal Windows folder picker), use LM Studio's folder, include subfolders, or open Hugging Face.
5. **Choose settings** (or keep the defaults) and click **Load and start chatting**; a bar shows the loading progress. Or click **Find the fastest settings**: tierlm loads the model, measures the settings that change speed on your PC, keeps the fastest and saves it for that model (about a minute, longer for big models). The first time on a GPU, the driver also prepares its programs once.

6 **Chat.** The side menu on the left holds the settings you can change while chatting, the API, a console, model info, saved chats and a benchmark: click an icon to open it, click again to fold it away. The right side shows speed, context, memory and, for mixture-of-experts models, which experts are working (the **?** explains the map). **Change setup** (top right) brings the setup screen back.

7 **Stop** by closing the black window.

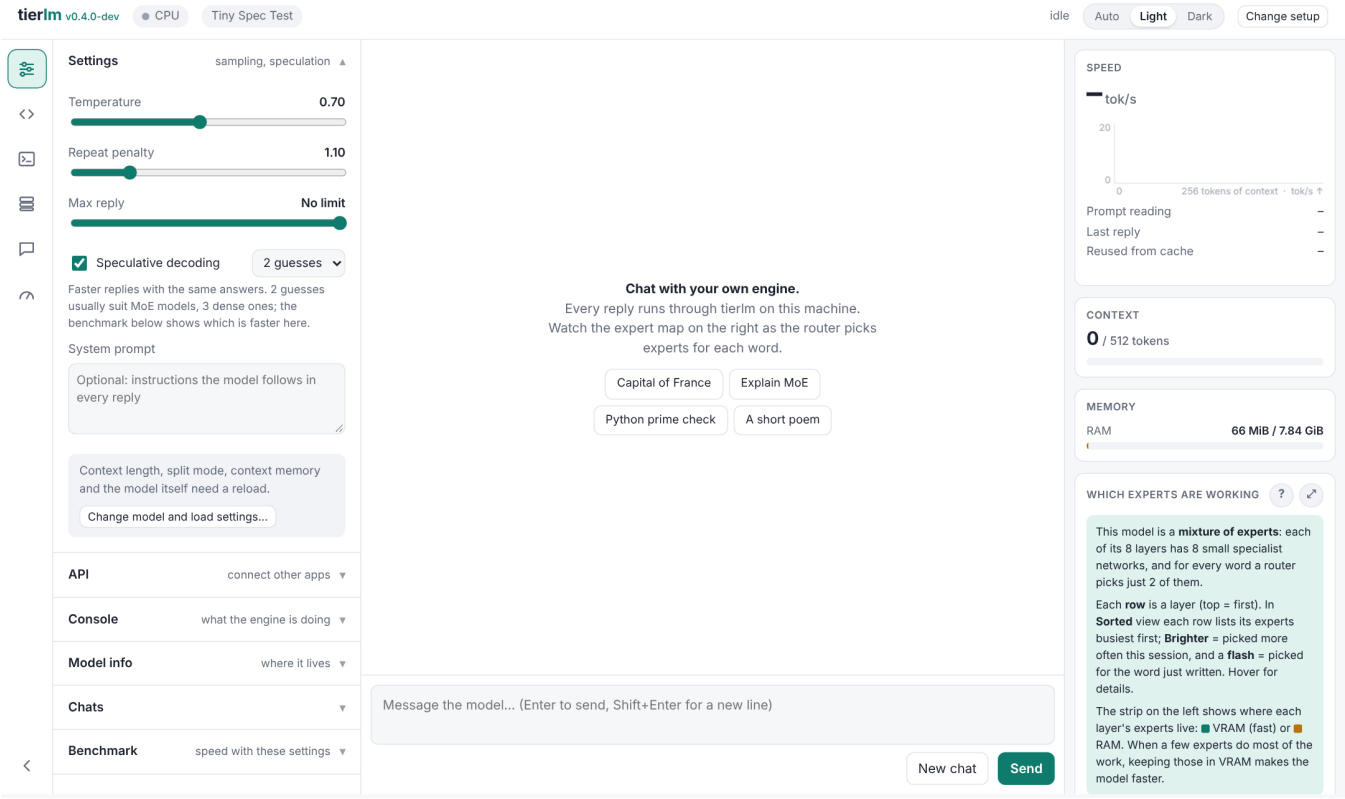



*Figure 2a. The chat screen: the side menu (Settings open), the conversation, and the live view with the explained expert map. Shown with a tiny test model.*

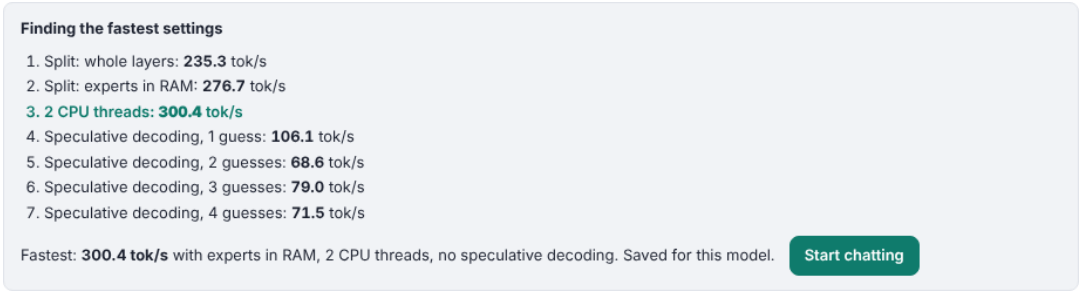



*Figure 2b. Find the fastest settings: each setting's speed as it's measured, then the winner (here on a stand-in GPU with a tiny test model).*

Where this model will go: It doesn't all fit on NVIDIA GeForce RTX 3070 Laptop GPU, so tierlm splits it: as many layers as fit (with their part of the context) go in VRAM, the rest in RAM, computed by the CPU. Expect speed between all-GPU and CPU-only.

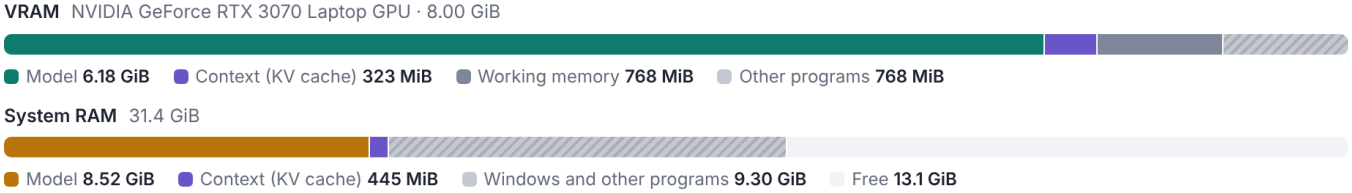



*Figure 2. The memory bars on the setup screen, for Qwen3-30B-A3B (14.7 GB) with 8K of context on an 8 GB RTX 3070: what goes in VRAM (model, context, working memory) next to what other programs use, and the rest in system RAM.*

**If a model is split although it should fit**, look at **Other programs** on the VRAM bar. Another tierlm window, LM Studio with a model loaded, or a game can hold most of the VRAM; tierlm says so under the bars. Close them and click **Rescan**.

**Tip:** tierlm remembers your models folder and, like LM Studio, your settings for each model, in `tierlm-settings.json`. Nothing leaves your computer: the page only talks to 127.0.0.1.

# 5. The settings

| Setting | What it does | Default |
|---|---|---|
| Context length | How much conversation the model remembers. Options go up to the model's own maximum, with the memory each size needs. | 4K tokens |
| CPU threads | Threads for work on the CPU. On laptops, fewer can be faster. | half your threads |
| Flash attention | Faster attention that reads the conversation memory once. Off only to compare or troubleshoot. | on |
| Context memory precision | How the conversation memory (KV cache) is stored. 8-bit uses half the memory of 16-bit with practically the same quality; 4-bit a quarter, with lower quality; 32-bit is exact but twice the size. 8-bit and 4-bit turn on flash attention. | 16-bit |
| Speculative decoding | For models with a draft layer (the model card says Speculative decoding, or a Draft add-on file sits next to it): faster replies with the same answers. Greyed out for other models. | off |
| Guesses per check | How many words the draft layer guesses before the model checks them. On the laptop, 3 was fastest for the dense Qwen3.6-27B and 2 for the MoE Qwen3.6-35B-A3B. Find the fastest settings measures it for you. The chat shows tokens per check after each reply: above about 1.5 it's helping. | 2 (MoE) or 3 (dense) |
| Prompt batch size | Tokens read at once when processing a prompt. Bigger is faster for long prompts, but uses more memory. | 256 |
| Active experts | Mixture-of-experts models only: experts used per layer per token. Fewer is faster, lower quality. | the model's own |
| Seed | -1 is random. Any other number makes replies repeatable. | -1 |
| When it doesn't fit | Automatic: experts in RAM for mixture-of-experts models (attention and context stay on the GPU; the CPU only reads the few experts each word picks), whole layers for dense ones (one GPU/CPU hand-off per word). Or pick either. | Automatic |
| Keep the busiest experts in VRAM | Mixture-of-experts models split with experts in RAM: put the experts this model has used most in VRAM instead of whole expert layers. Learned as you chat (only counts, kept next to the settings file); Find the fastest settings learns them right away and turns this on only if it's faster on your PC. | off |
| VRAM limit | Use at most this much VRAM. Keeps room for other apps, or simulates a smaller card. | 0 (all free) |
| Force CPU | Ignore the GPU. Useful for troubleshooting. | off |
| Temperature | In the chat panel. 0 always picks the most likely word; higher is more varied. | 0.70 |
| Repeat penalty | In the chat panel. Makes recently used words less likely, which stops loops. | 1.10 |
| Max reply | In the chat panel. The longest a reply can be. No limit lets the model finish; thinking models such as Qwen3.x often use thousands of tokens before they answer. | No limit |
| Top-k / Top-p / Min-p | Which candidate words are considered. Same meaning and defaults as LM Studio. | 40 / 0.90 / 0.05 |

# 6. Connect other apps

tierlm speaks the same API as OpenAI, so apps made for OpenAI (or for LM Studio and Ollama) can use the model loaded in tierlm. On the setup screen (**Change setup** from the chat), the **Connect other apps** card shows the address to give them, the **Allow other devices** switch and the **API key**.

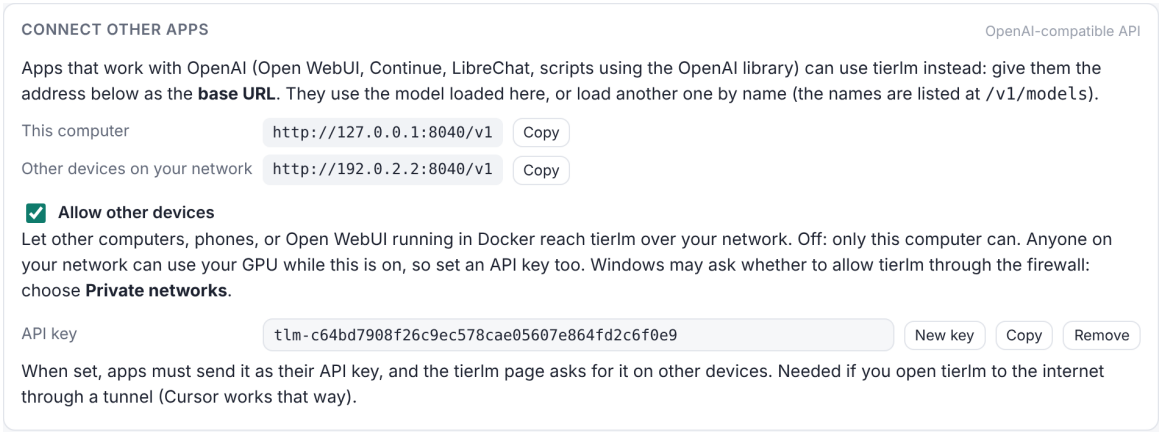



*Figure 3. Connect other apps, with Allow other devices on and an API key set.*

| App | How to connect |
|---|---|
| Apps on this computer | Base URL http://127.0.0.1:8040/v1, any text as the API key unless you set one. |
| Other PCs and phones | Turn on Allow other devices, then use the second address shown (http://<this PC>:8040/v1). If Windows asks about the firewall, choose Private networks. Set an API key. |
| Open WebUI | Admin Panel → Settings → Connections → add an OpenAI API connection. In Docker on this PC: http://host.docker.internal:8040/v1, with Allow other devices on (Docker's traffic doesn't count as this computer). tierlm's models appear in Open WebUI's list; picking one loads it. |
| Continue (VS Code, JetBrains) | A model with provider: openai, apiBase: http://127.0.0.1:8040/v1, any model name, and the API key if set. |
| Cursor | Cursor calls custom models from its own servers, so it needs a public HTTPS address from a tunnel (for example cloudflared tunnel --url http://localhost:8040). Turn on Allow other devices and set an API key first. Chat works; agent mode needs tool calling, which isn't built yet. |
| Scripts | The OpenAI library: OpenAI(base_url="http://127.0.0.1:8040/v1", api_key="none"). Streaming, stop strings, seeds and token counts work. |

**Who can connect.** With Allow other devices off (the default), only this computer can. With an API key set, every app must send it, and so must the tierlm page on other devices (it asks once). Traffic through a tunnel never counts as this computer, so a tunnel can't skip the key. The switch and the key can only be changed on the computer running tierlm.

# 7. How it works




*Figure 4. How one word is produced. tierlm repeats this for every word of a reply.*

Each model family is a variation on the same transformer design. tierlm describes the differences with a handful of switches (how positions are encoded, whether attention is normalised, whether the feed-forward part is one network or many experts, how expert weights are combined), so one graph builder runs all of them. The tokenizer and chat format are read from the model file.

## Mixture-of-experts (MoE) models

In an MoE model, each layer has many small "expert" networks (128 in Qwen3-30B-A3B) and a router picks a few of them (8) for each word. So a 30-billion-parameter model only does about 3 billion parameters of work per word. The experts are most of the file, but each one is rarely needed. That makes MoE models the best case for tiering: the parts used for every word go in fast memory, and the experts can wait in slower memory.

## The tier loader

When a model doesn't fit in VRAM, tierlm decides where every tensor goes before loading. In the default **whole layers** mode, it fills VRAM from the last layer backwards (each layer with its share of the conversation memory, the KV cache), plus the output layer. The first layers stay in RAM and run on the CPU. Each word therefore crosses from CPU to GPU once.

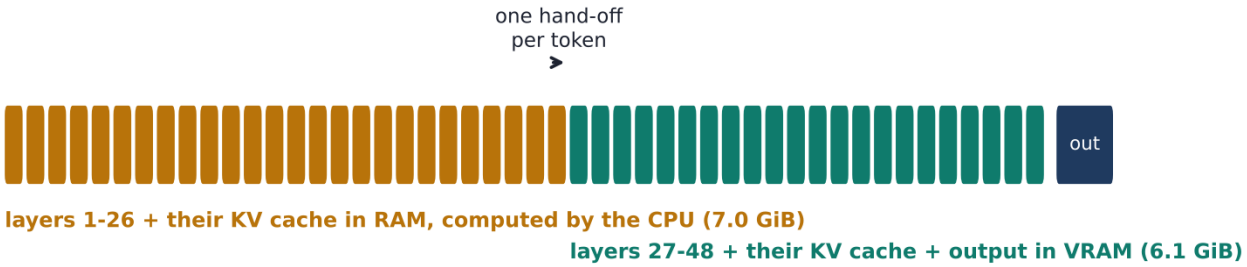



*Figure 5. How Qwen3-30B-A3B (14.7 GB) was placed on an 8 GB RTX 3070: 22 of 48 layers in VRAM.*

## Finding the hidden hand-offs

The first tiered version was slower than running on the CPU alone. A profiler built for this showed why: ggml's scheduler sends any step without weights of its own (normalisation, attention over the conversation memory) to the GPU if it can, even in the middle of a layer that lives in RAM. That meant 80 to 137 trips between CPU and GPU per word, and on Windows each trip waits for the GPU. tierlm now pins every step of a RAM layer to the CPU, and the trips dropped to 2.

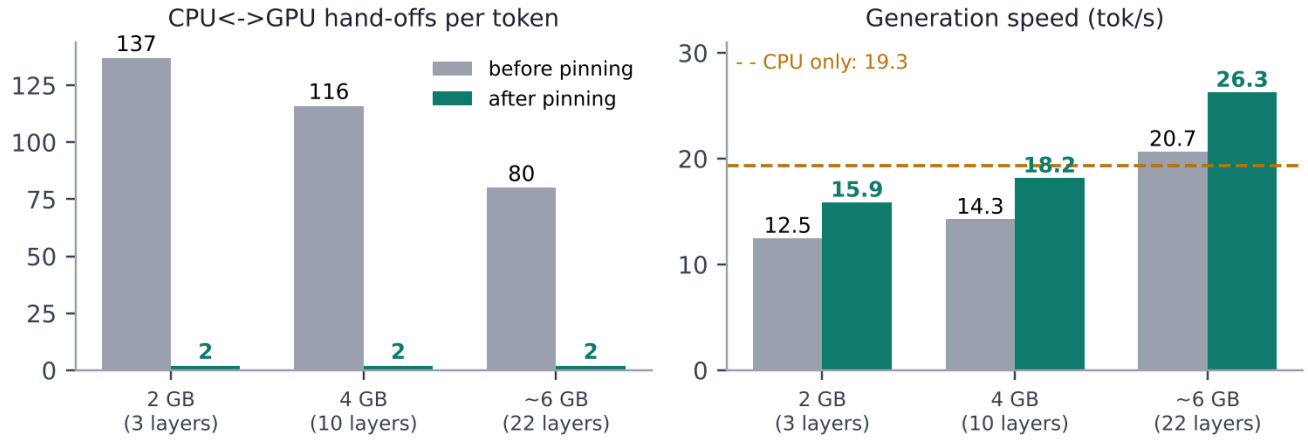



*Figure 6. Qwen3-30B-A3B on the RTX 3070 at three VRAM sizes, before and after pinning RAM layers to the CPU.*

## Speculative decoding

When part of a dense model is in RAM, every word means reading all of that part from RAM once, so RAM speed sets the pace: about 3 words a second for a 27B model on a laptop. Speculative decoding gets several words out of each read. The model's small draft layer guesses the next few words (up to **Guesses per check**); the model then works on its own word and all the guesses in one pass, which costs about the same as one word because the weights are read once. Guesses the model agrees with are kept for free; at the first one it disagrees with, it takes its own word and the rest are undone. Every word that comes out is the one the model itself would have picked, so the answers don't change, only the speed.

Undoing guesses is easy for normal attention layers (they forget the extra positions) but not for the Gated DeltaNet layers in Qwen3.5 / 3.6, whose running memory can't be rewound. tierlm keeps a snapshot of that memory after every guessed word during the check and restores the right one, which tests show is exact.

## Hot experts

Some experts are picked far more than others. tierlm counts each model's picks as you chat and can fill spare VRAM with the most-used experts of any layer, instead of whole expert layers. A layer whose experts are partly in VRAM is computed in two parts, the GPU's and the CPU's, and each pick is counted only in the part that holds it. The CPU skips picks of experts that are in VRAM (one of tierlm's two small changes to ggml), so it only works on the experts in RAM. On the laptop the experts in VRAM did 71% of the expert work, but the extra copying and bookkeeping per layer still made it a little slower than the plain split, so it's measured per PC and off by default.

## CUDA graphs

Timing each word showed the laptop's GPU spending about 22 ms per word on a part of the model it can read in 2 to 3 ms: it was busy launching thousands of small jobs, which is slow on Windows. ggml can record a word's GPU jobs once and replay them with one launch (CUDA graphs), but that's off unless turned on, and a recording only replays while the work keeps exactly the same shape. tierlm turns it on and keeps the shape steady: new conversation memory is written by position, attention covers the memory in steps of 256 (the unused part masked out, so results don't change), and each kind of step keeps its own recording. Generation got about 50% faster.

## Using the whole CPU

A CPU engine built for one generation either can't use newer instructions or won't start on older processors. tierlm's download carries one engine per x86 generation and loads the best one the processor supports. On load, weights that will run on the CPU are also rearranged into the interleaved layout those wide instructions read fastest ("repacking"), which made prompt reading on the CPU about 3.5× faster.

# 8. Test results

## Correctness first

Speed only counts if the answers are right. Every change was checked against references that don't share any code with tierlm:

| Check | How | Result |
|---|---|---|
| Model math, every architecture | Tiny random models of each family, compared with a plain NumPy implementation written from the architecture's definition. Exact (f32) and normal (f16) conversation memory, one-shot and word-by-word, prompts over 256 tokens. | All 7 variants match to about 2e-6 |
| Splitting between GPU and RAM | The same checks with the model split at 5 VRAM sizes in both split modes, using a stand-in GPU (ggml's RPC backend). | 200 of 200 pass |
| Hybrid Qwen3.5 / 3.6 / 3.8 | A NumPy version of Hugging Face's Qwen3.5 model (Gated DeltaNet + gated attention), dense and MoE (routed experts plus a gated shared expert, and the newer combined gate/up expert layout), on tiny models written the way the converter writes them, also split between GPU and RAM. Deliberately wrong file conventions and a broken shared expert were checked to fail. | Matches to about 1e-5 |
| On the GPU | The same models on CUDA (RTX 3070) with normal settings. CUDA uses reduced-precision (TF32) tensor cores for small batches on RTX 30-series and newer, so results differ by a few %; the top prediction must match. | All pass; same top prediction |
| Several GPUs | Every model type split over two stand-in GPUs (ggml's RPC backend) and RAM, in both split modes at many memory sizes, including speculative decoding. | All match the reference |
| Hot experts | Every mixture-of-experts type with made-up usage counts, so layers end up whole, split or absent in VRAM, at many VRAM sizes, word by word and with 300-word prompts. | All match (about 1e-6) |
| Speculative decoding | Normal generation against speculative generation at temperature 0 on every model type, with the real draft layer and with scripted guesses where every 3rd or 5th is wrong, over several turns, also split between GPU and RAM. Undoing guesses checked to be exact at every depth. | 140 of 140 identical |
| Repacking for the CPU | Q4_0 models with and without CPU repacking. | Same results (to 3e-6); prompts 3.5× faster |
| OpenAI-compatible API | OpenAI's own Python library against tierlm: model list, replies streamed and not, stop strings, token counts, loading a model by name. Plus the API key and network rules. | All pass |
| Models in several files | Every family written as a 3-part split GGUF by Hugging Face's gguf library, loaded through part 1. | All 7 match the reference |
| Tokenizers (logic) | Small tokenizers trained with Hugging Face's tokenizers and Google's sentencepiece libraries, 31 tricky strings each (capitals, numbers, emoji, Cyrillic, Japanese, Windows line endings). | All exact, incl. both SentencePiece storage formats |
| Tokenizers (real models) | 29 strings against each model's official Hugging Face tokenizer. | Qwen3-0.6B, Llama-3.2-1B and TinyLlama: 29/29 each |

| Check | How | Result |
|---|---|---|
| Real weights | "The capital of France is" on real models, top prediction. | " Paris": Qwen3-0.6B 65%, Llama-3.2-1B 68%, TinyLlama 37%, Qwen3-30B-A3B 69-73% (CPU, CUDA, split) |

## Intel Arc 140V laptop (Vulkan)

Intel Core Ultra 7 258V with Arc 140V integrated graphics, 32 GB shared memory, Windows 11. Model: OLMoE-1B-7B-Instruct Q4_K_M (3.9 GB). Speed falls as the conversation grows, because each new word looks back over all earlier ones; the f16 conversation memory and flash attention made long conversations much faster.

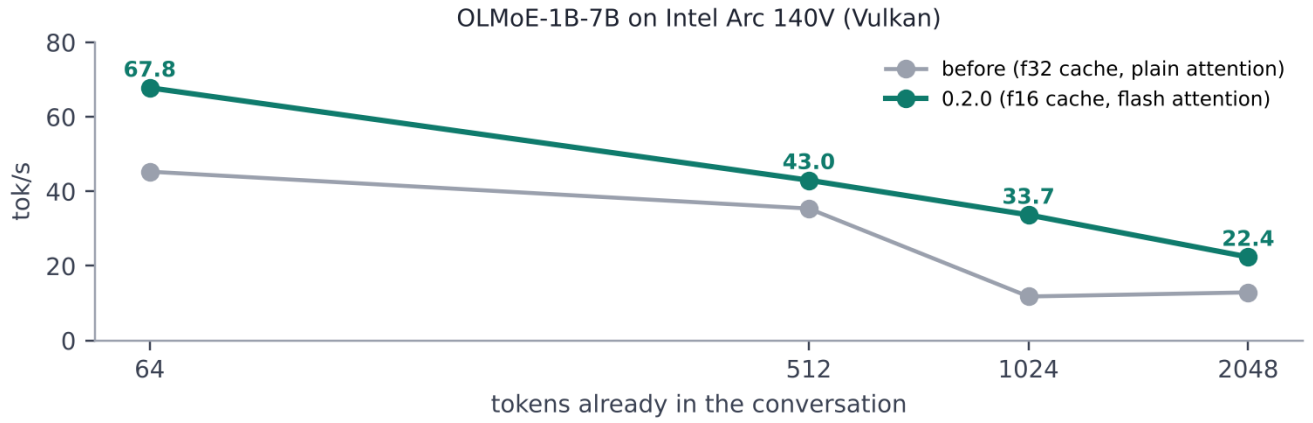



*Figure 7. Generation speed against conversation length on the Arc 140V. For comparison, the same laptop's CPU (under WSL) ran it at about 46 tok/s at short context.*

## RTX 5090 desktop (CUDA)

Measured with 0.4.0, before CUDA graphs; 0.5.0 results will be added. AMD Ryzen 9 9950X3D, NVIDIA GeForce RTX 5090 (32 GB), 64 GB DDR5, Windows 11, using the release zip with nothing else installed (a new-user test). Model: Qwen3.6-35B-A3B Q8_0, a 46 GB file from LM Studio's folder, with 128K tokens of context. It doesn't fit in 32 GB of VRAM, so tierlm split it between VRAM and RAM by itself.

| Qwen3.6-35B-A3B Q8_0, 46 GB | Placement | Generation |
|---|---|---|
| CUDA, all free VRAM, 128K context | split between VRAM and RAM | 36.6 tok/s |

## RTX 3070 laptop (CUDA and Vulkan)

AMD Ryzen 9 5900HX, NVIDIA GeForce RTX 3070 Laptop GPU (8 GB), 32 GB RAM, Windows 11, CUDA 13.4. Generation speed over 100-200 words.

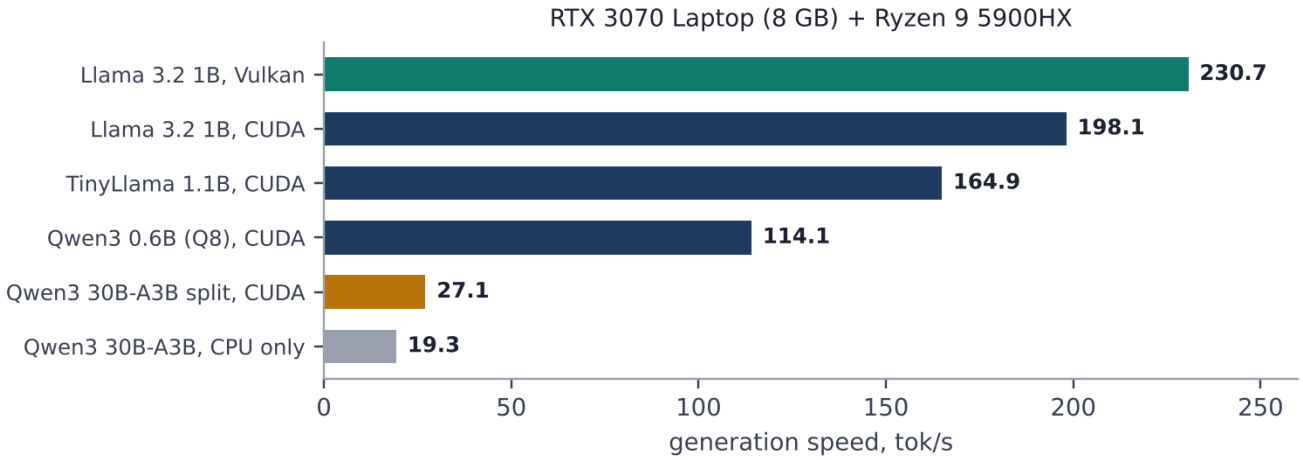



*Figure 8. Small models run entirely in VRAM; Qwen3-30B-A3B (14.7 GB) is split between VRAM and RAM.*

| Qwen3-30B-A3B Q3_K_M, 14.7 GB | Placement | Generation | Prompt |
|---|---|---|---|
| CPU only | all in RAM | 19.3 tok/s | 32 tok/s |
| CUDA, VRAM limited to 2 GB | 3 of 48 layers in VRAM | 15.9 tok/s | 24 tok/s |
| CUDA, VRAM limited to 4 GB | 10 of 48 layers in VRAM | 18.2 tok/s | 25 tok/s |
| CUDA, all free VRAM | 22 of 48 layers (6.1 GB) in VRAM | 26.3 tok/s | 33 tok/s |
| CUDA, all free VRAM, 6 CPU threads | 22 of 48 layers in VRAM | 27.1 tok/s | 31 tok/s |
| Vulkan, all free VRAM | 22 of 48 layers in VRAM | 16.1 tok/s (first run, driver still preparing; to be re-measured) | 2 tok/s |

## Speculative decoding on the RTX 3070 laptop

Qwen3.6-27B (Unsloth's MTP Q4_K_M, 17 GB, a dense model) doesn't fit in 8 GB: 23 of its 64 layers went to VRAM and the rest to RAM, with 16K of context at 8-bit. With 3 guesses per check, each check produced 2.6 words on average and 53% of guesses were kept.

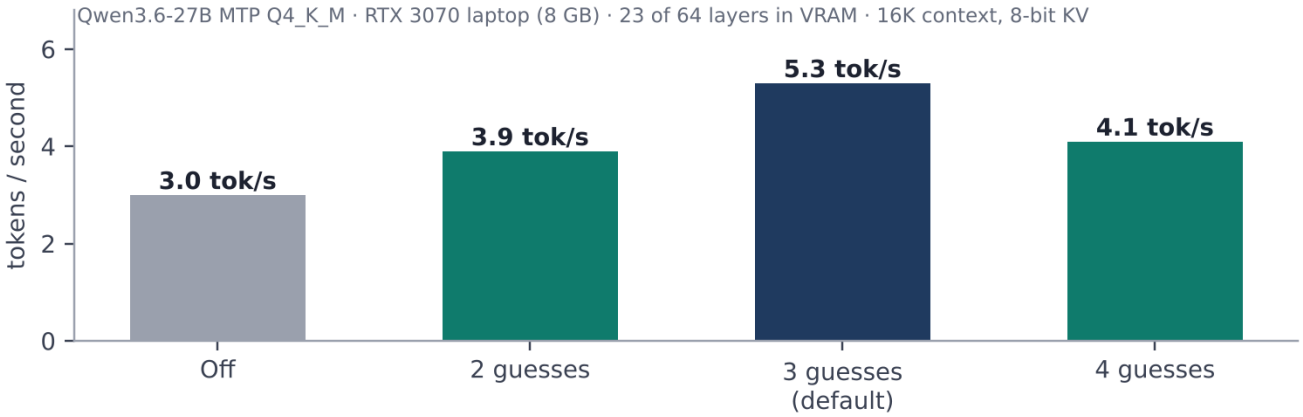



*Figure 9. Generation speed with Speculative decoding off and with 2, 3 and 4 guesses per check. More guesses stop paying once the later ones are rarely kept, so 3 is the default for dense models. Desktop results with an RTX 5090 will be added.*

## Mixture-of-experts on the RTX 3070 laptop

Qwen3.6-35B-A3B (Unsloth's MTP UD-Q3_K_XL, 17 GB) uses about 3 billion of its 35 billion parameters per word. Keeping its attention and context on the GPU with only the experts in RAM is 2.5 times faster than splitting by whole layers. CUDA graphs made every GPU-heavy setting faster; whole layers barely changed, because there the CPU does most of the work. With the GPU part this quick, speculative decoding adds less than before: each guessed word the model checks pulls in its own experts from RAM, and RAM is now the limit. Measured by Find the fastest settings (short replies at temperature 0), so a normal chat comes out somewhat lower; back-to-back runs vary by 1 to 2 tokens/second as the laptop warms up.

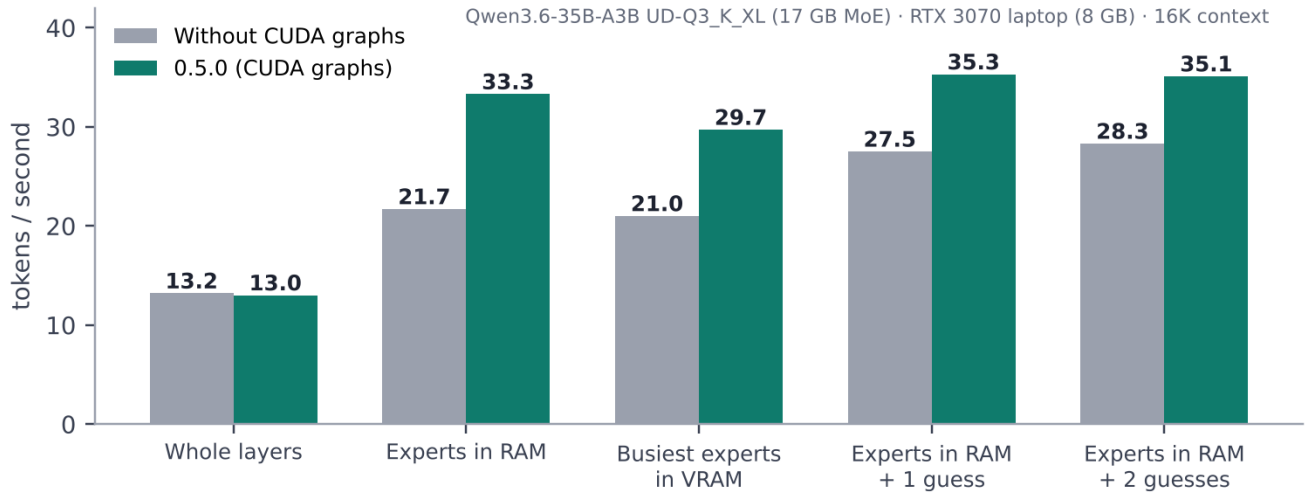



*Figure 10. The same model and laptop, each setting without and with CUDA graphs.*

**What the numbers say.** With 22 of 48 layers on the GPU, the 30-billion-parameter model runs about 40% faster than on the CPU alone. On this laptop the CPU part runs about 25% slower whenever the GPU is busy. CUDA loaded with no GPU work was not slower, which points to the CPU and GPU sharing the laptop's power budget, so desktops should gain more. For a model that fits in VRAM, Vulkan was 16% faster than CUDA once its programs were prepared (230.7 against 198.1 tok/s).

# 9. Troubleshooting

| Problem | What to do |
|---|---|
| "Windows protected your PC" | Click More info, then Run anyway. You can check the zip against the .sha256 file on the Releases page first. |
| The GPU isn't listed, or a yellow box mentions VK_ICD_FILENAMES / VK_DRIVER_FILES | Leftover settings from an old driver install point Vulkan at a driver file that no longer exists. Run the reg delete command shown in the yellow box, then sign out of Windows and back in. Tell-tale sign: Vulkan works as administrator but not as your normal account. |
| The GPU still isn't listed | Update the graphics driver. Run tierlm-devices.exe in the tierlm folder: it lists what tierlm can see and why a GPU failed. |
| The page shows a different app, or "404" | Another program uses the port. Start tierlm on another one: start-tierlm.bat 8050, then open http://127.0.0.1:8050. Use 127.0.0.1, not localhost. |
| The page still shows an old version | An older tierlm is still running in another window and owns the port. Close every tierlm window and start again. |
| "No models found" | Put a .gguf file in the models folder, or use ⋯ → Choose models folder..., then Rescan. |
| A model is "not supported yet" | The label says which architecture or tokenizer the file uses. See section 3. A file labelled Vision add-on (mmproj) is not a model on its own. |
| Replies repeat or ramble | Raise Repeat penalty a little (e.g. 1.15). Very small models loop more easily, and can state wrong things confidently. |
| Speculative decoding is greyed out | The model file has no draft layer. Some publishers (Unsloth) put it in a separate mtp-<model>.gguf: download it into the same folder and click Rescan. |
| The side menu disappeared | It's folded away: click the arrow at the bottom left, or any icon on the left edge. |
| It's slow | Try Find the fastest settings on the setup screen. Check the Model info section: a model split with RAM is slower than all-VRAM. Try a smaller quantization, a shorter context, or fewer CPU threads on laptops. |
| "not enough memory" | Pick a shorter context, close other programs, or pick a smaller model. The SSD tier isn't built yet, so model + context must fit in VRAM + RAM. |
| A model is split although it should fit | Look at Other programs on the VRAM bar: another tierlm window, LM Studio or a game is holding VRAM. Close it and click Rescan. |
| Another device or Open WebUI can't connect | Turn on Allow other devices in Connect other apps and use the network address it shows, not 127.0.0.1. If you cancelled the firewall question, allow tierlm-server for Private networks in Windows Security → Firewall & network protection. |

# 10. Building from source

Only needed to change tierlm or build a release. On Windows: Visual Studio 2022 or 2026 with Desktop development with C++, the Vulkan SDK, and for CUDA the CUDA Toolkit. Then, in a Developer Command Prompt:

```
git clone --recurse-submodules https://github.com/EHTi-Chad/tierlm.git
cd tierlm
cmake -S . -B build -G Ninja -DTIERLM_VULKAN=ON -DTIERLM_CUDA=ON
cmake --build build
build\tierlm-devices.exe
```

`scripts\package-windows.bat 0.5.0 both` builds both release zips (a CPU engine for every x86 generation, C runtime included; the full one also bundles NVIDIA's CUDA runtime). docs/BUILD.md covers Linux, WSL, every test, and the pitfalls hit during development (an accidental Debug build, stale Vulkan settings, ports in use, the CUDA compiler flag).

# 11. Known limits and what's next

- **Hot experts, next steps**: make the split cheaper (fewer copies per layer) and let the GPU and CPU work on the same layer at once, which needs changes to ggml's scheduler. Move experts while the model is loaded as usage shifts.
- **Find the fastest settings, measured like a real chat**: with your sampling settings and longer replies, so its numbers match what you see.
- **SSD tier**: not built yet. Model and context must fit in VRAM plus RAM.
- **More families**: Gemma 4, Phi, DeepSeek, gpt-oss; and real-weight tests of Qwen3-Coder-Next, Mixtral and Qwen2.5.
- **Tool calling** in the API, for coding agents (Cursor's agent mode, Cline, Continue's agent mode).
- **More speed**: prompt-lookup guesses for models without a draft layer (good for code and edits), and the AVX-512 Zen 4 and AMX engines (which need a different compiler on Windows).
- **Qwen3 thinking** is shown as raw <think> text for now.
- **Several GPUs** must be the same kind (all CUDA, or all Vulkan); tested with stand-in GPUs, real two-card results welcome. AMD GPU testing, and a ROCm build for Radeon cards.

tierlm is MIT-licensed. ggml (with tierlm's small changes, docs/GGML_CHANGES.md), cpp-httplib and nlohmann/json keep their own MIT licences (THIRD_PARTY_NOTICES.md). Model weights are not included and have their own licences.